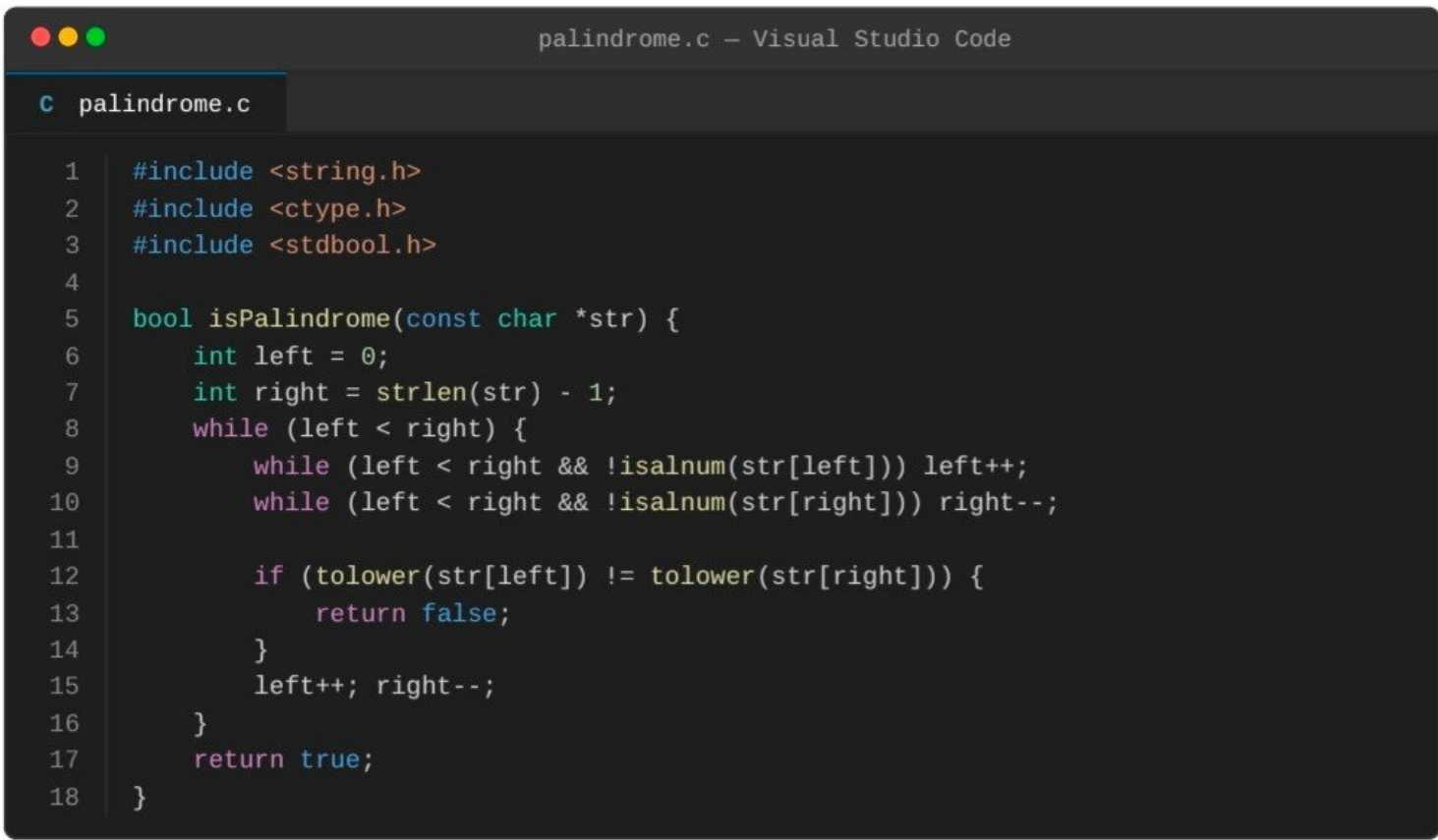


1. Palindrome Validation

Checks if a string is identical forwards and backwards while ignoring case and non-alphanumeric symbols using an optimized two-pointer walk.



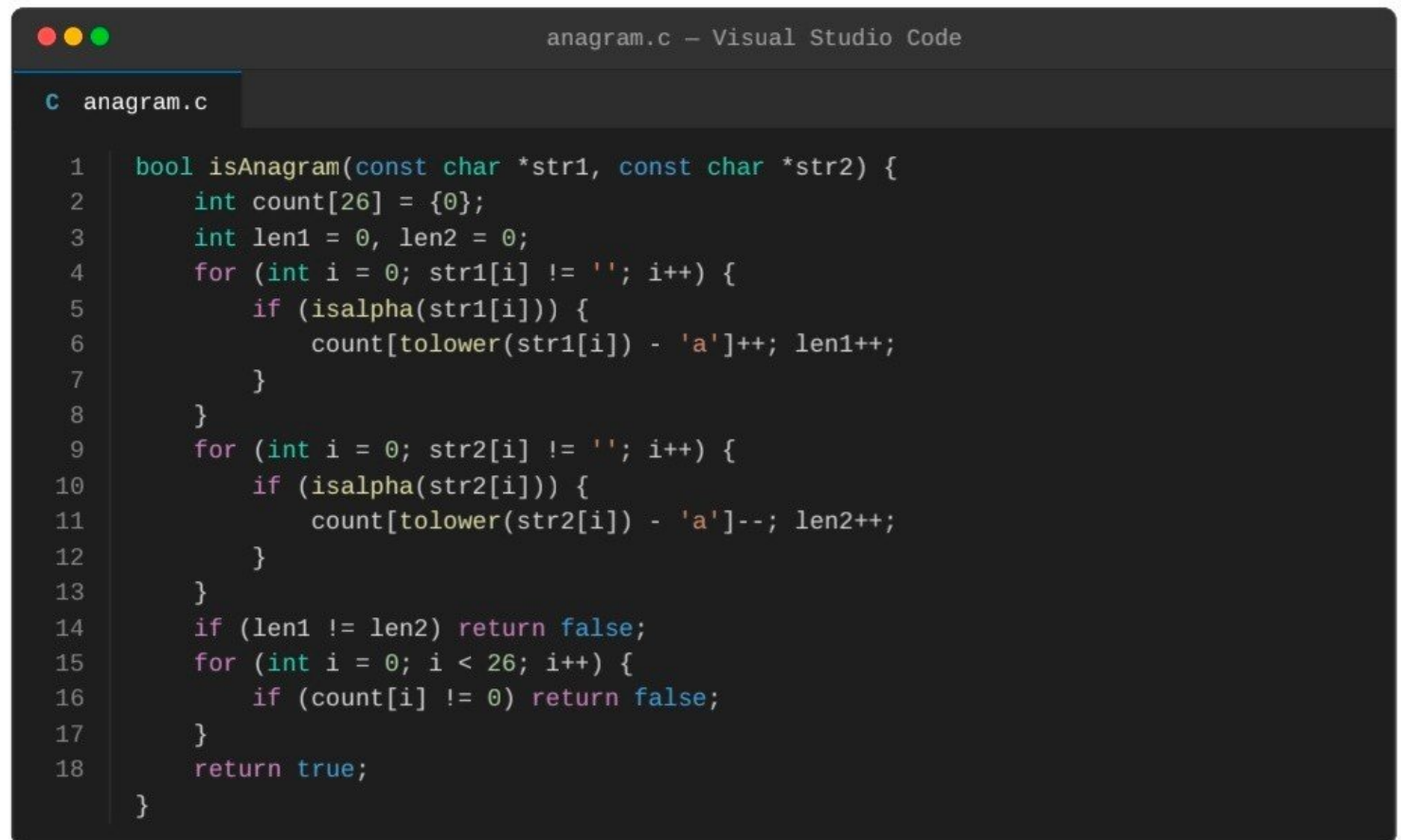
```
palindrome.c - Visual Studio Code

C palindrome.c

1  #include <string.h>
2  #include <ctype.h>
3  #include <stdbool.h>
4
5  bool isPalindrome(const char *str) {
6      int left = 0;
7      int right = strlen(str) - 1;
8      while (left < right) {
9          while (left < right && !isalnum(str[left])) left++;
10         while (left < right && !isalnum(str[right])) right--;
11
12         if (tolower(str[left]) != tolower(str[right])) {
13             return false;
14         }
15         left++; right--;
16     }
17     return true;
18 }
```

2. Anagram Check

Uses a uniform hash-frequency counter map to establish structural letter equality across two given character arrays.



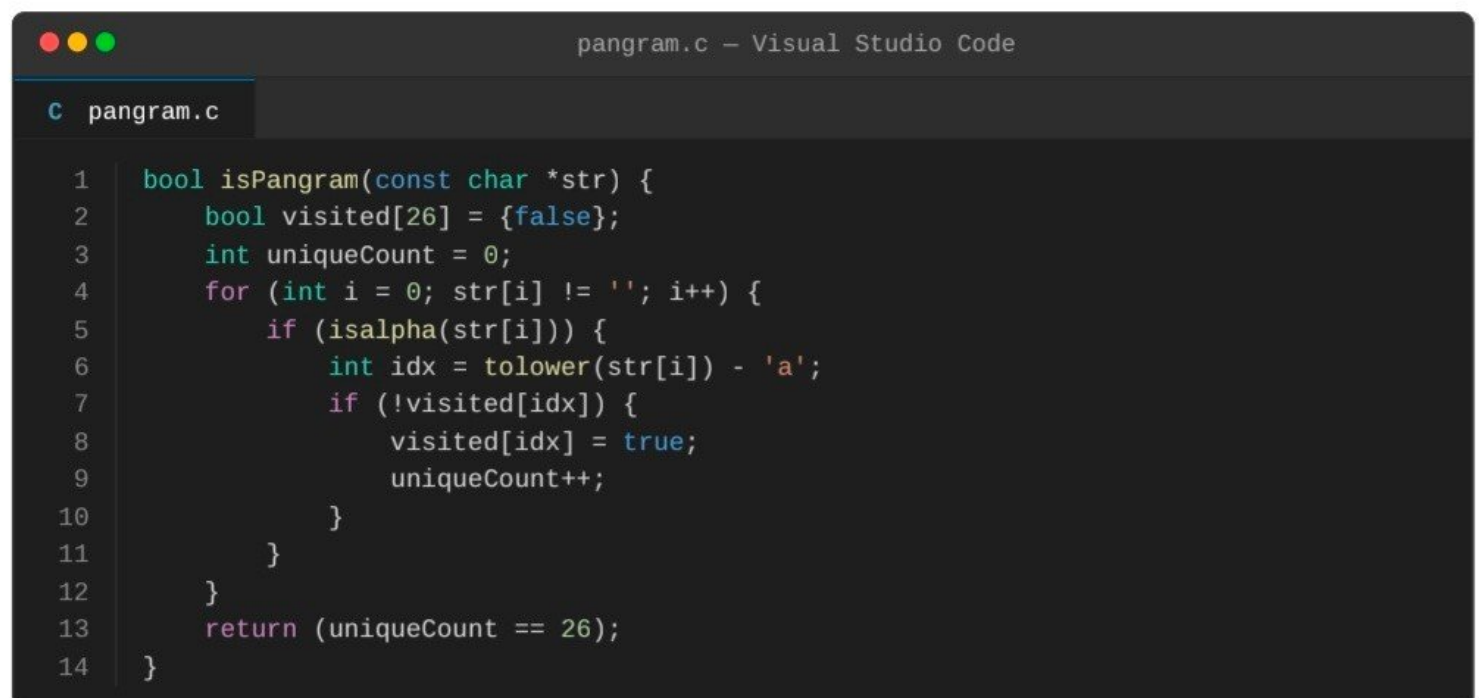
```
anagram.c - Visual Studio Code

C anagram.c

1  bool isAnagram(const char *str1, const char *str2) {
2      int count[26] = {0};
3      int len1 = 0, len2 = 0;
4      for (int i = 0; str1[i] != '\0'; i++) {
5          if (isalpha(str1[i])) {
6              count[tolower(str1[i]) - 'a']++; len1++;
7          }
8      }
9      for (int i = 0; str2[i] != '\0'; i++) {
10         if (isalpha(str2[i])) {
11             count[tolower(str2[i]) - 'a']--; len2++;
12         }
13     }
14     if (len1 != len2) return false;
15     for (int i = 0; i < 26; i++) {
16         if (count[i] != 0) return false;
17     }
18     return true;
19 }
```

3. Pangram Detection

Verifies whether a sentence contains at least one occurrence of every individual character from the English alphabet standard set.



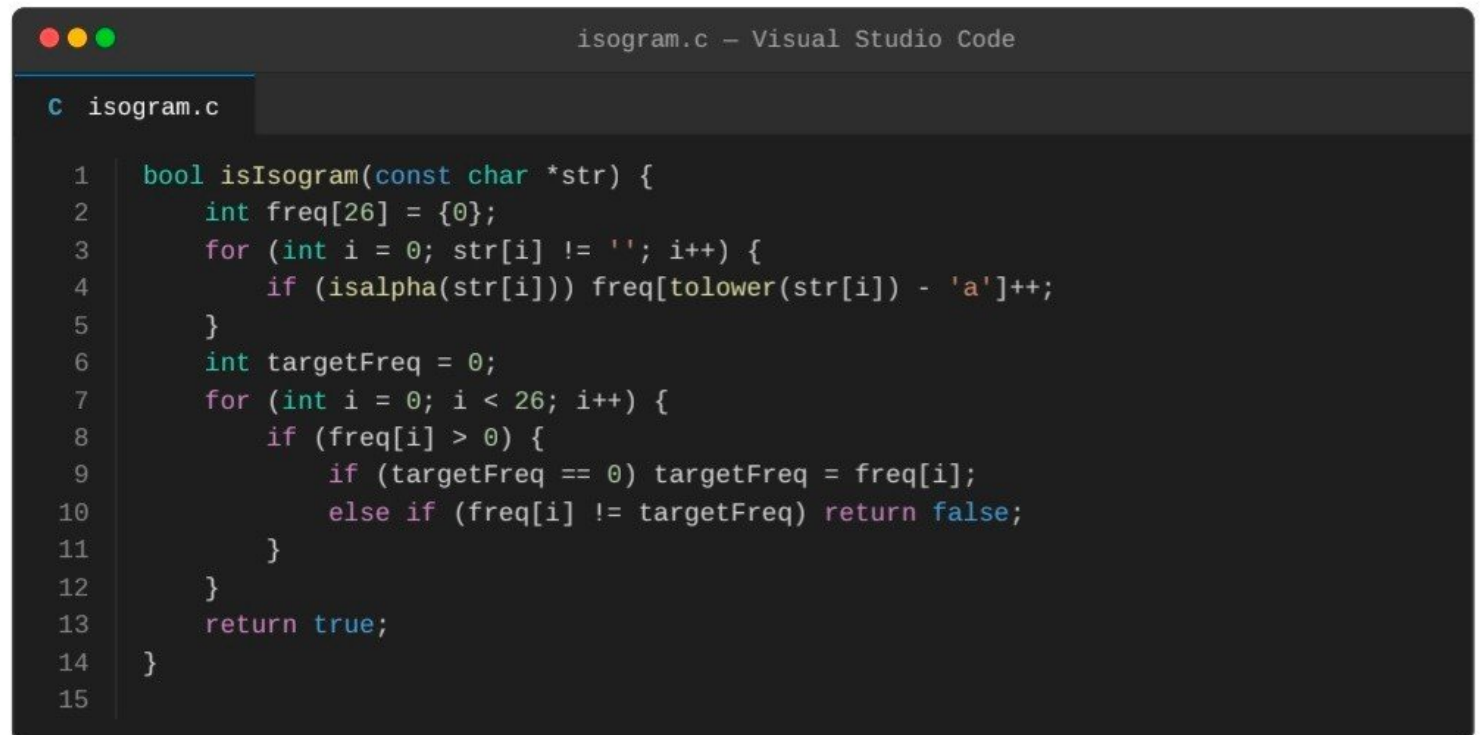
```
pangram.c - Visual Studio Code

C pangram.c

1  bool isPangram(const char *str) {
2      bool visited[26] = {false};
3      int uniqueCount = 0;
4      for (int i = 0; str[i] != '\0'; i++) {
5          if (isalpha(str[i])) {
6              int idx = tolower(str[i]) - 'a';
7              if (!visited[idx]) {
8                  visited[idx] = true;
9                  uniqueCount++;
10             }
11         }
12     }
13     return (uniqueCount == 26);
14 }
```

4. Isogram Evaluation

Evaluates if every single character present inside the string occurs an exact equal count of times relative to one another.



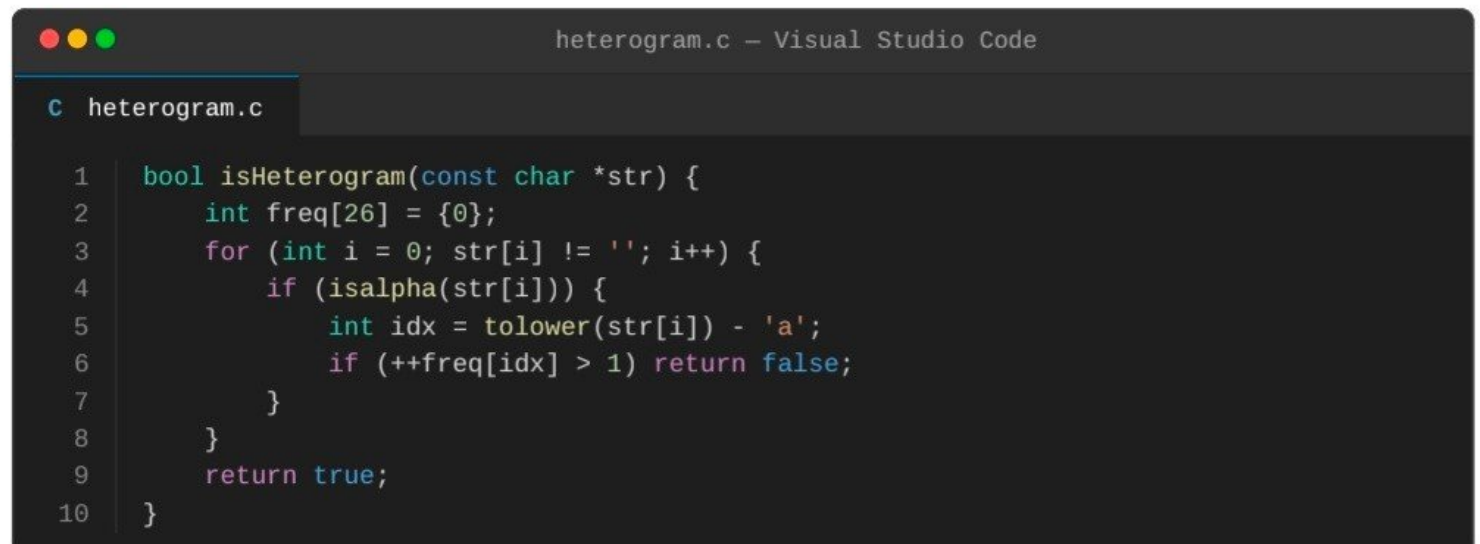
```
isogram.c - Visual Studio Code

C isogram.c

1  bool isIsogram(const char *str) {
2      int freq[26] = {0};
3      for (int i = 0; str[i] != '\0'; i++) {
4          if (isalpha(str[i])) freq[tolower(str[i]) - 'a']++;
5      }
6      int targetFreq = 0;
7      for (int i = 0; i < 26; i++) {
8          if (freq[i] > 0) {
9              if (targetFreq == 0) targetFreq = freq[i];
10             else if (freq[i] != targetFreq) return false;
11         }
12     }
13     return true;
14 }
15
```

5. Heterogram Isolation

A variation of an isogram where the frequency ceiling threshold for every letter is strictly locked to one.



```
heterogram.c - Visual Studio Code

C heterogram.c

1  bool isHeterogram(const char *str) {
2      int freq[26] = {0};
3      for (int i = 0; str[i] != '\0'; i++) {
4          if (isalpha(str[i])) {
5              int idx = tolower(str[i]) - 'a';
6              if (++freq[idx] > 1) return false;
7          }
8      }
9      return true;
10 }
```

6. Semordnilap Pairing

Determines if a target word creates a brand new alternative valid word when processed backwards (e.g. "stressed" and "desserts").

A screenshot of the Visual Studio Code editor interface. The title bar at the top reads "semordnilap.c - Visual Studio Code". The editor window shows a C program named "semordnilap.c". The code is as follows:

```
1  bool isSemordnilap(const char *s1, const char *s2) {
2      int len1 = strlen(s1);
3      if (len1 != (int)strlen(s2) || strcmp(s1, s2) == 0) return false;
4      for (int i = 0; i < len1; i++) {
5          if (tolower(s1[i]) != tolower(s2[len1 - 1 - i])) {
6              return false;
7          }
8      }
9      return true;
10 }
```

7. Acronym Generation

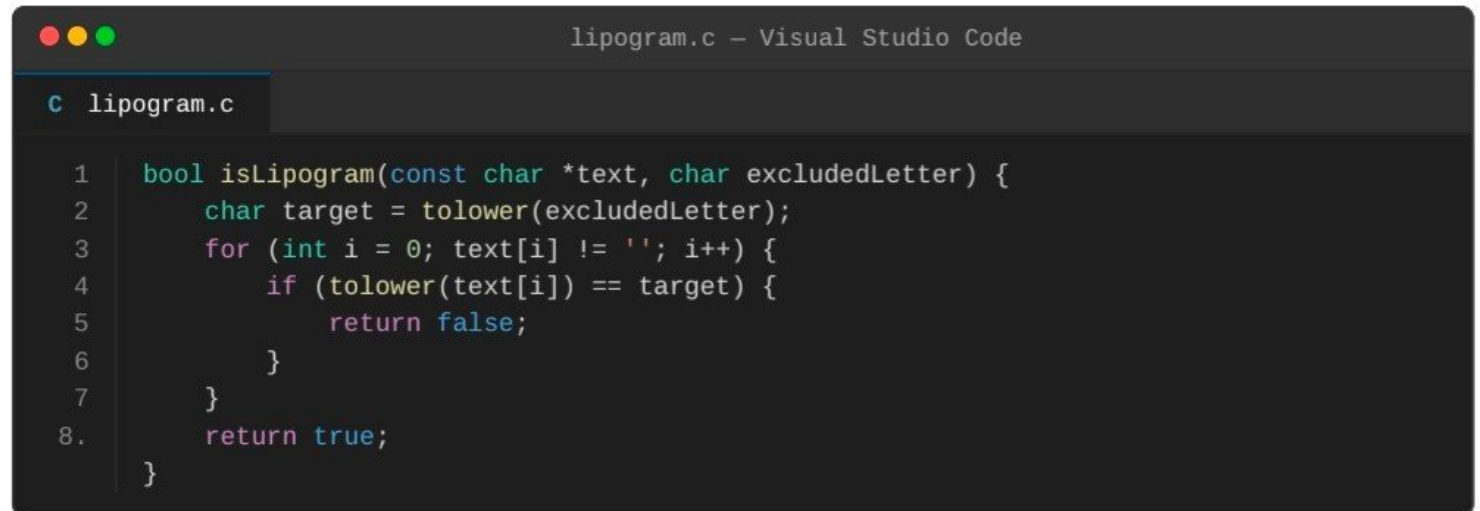
Iterates through string phrases to extract uppercase initials from each isolated whitespace-bounded text cluster.

A screenshot of the Visual Studio Code editor interface. The title bar at the top reads "acronym.c - Visual Studio Code". The editor window shows a C program named "acronym.c". The code is as follows:

```
1  void makeAcronym(const char *phrase, char *result) {
2      int count = 0;
3      bool newWord = true;
4      for (int i = 0; phrase[i] != '\0'; i++) {
5          if (isspace(phrase[i])) {
6              newWord = true;
7          } else if (isalpha(phrase[i]) && newWord) {
8              result[count++] = toupper(phrase[i]);
9              newWord = false;
10         }
11     }
12     result[count] = '\0';
13 }
```

8. Lipogram Verification

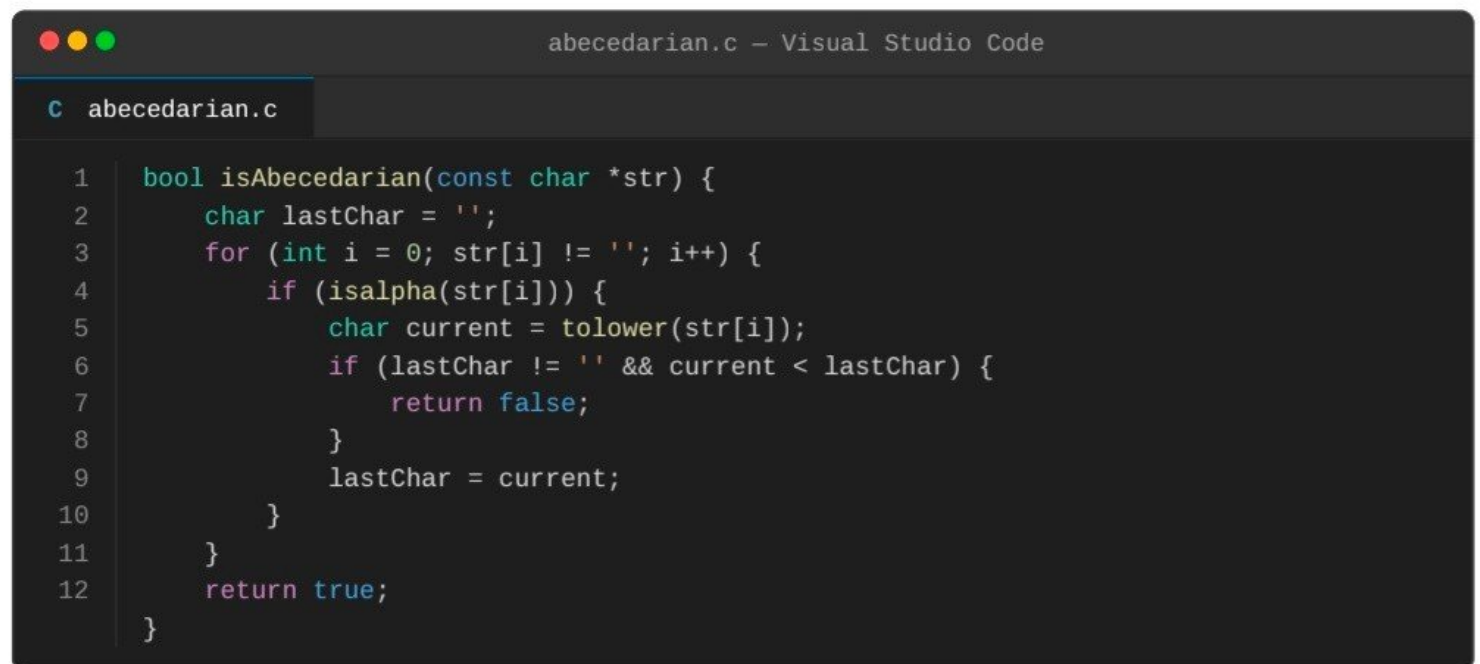
Checks strings to verify that a designated banned letter is missing entirely from the text stream.

A screenshot of the Visual Studio Code editor interface. The title bar at the top reads "lipogram.c - Visual Studio Code". The editor window shows a C program named "lipogram.c". The code is as follows:

```
1  bool isLipogram(const char *text, char excludedLetter) {
2      char target = tolower(excludedLetter);
3      for (int i = 0; text[i] != '\0'; i++) {
4          if (tolower(text[i]) == target) {
5              return false;
6          }
7      }
8      return true;
9  }
```

9. Abecedarian Sequence

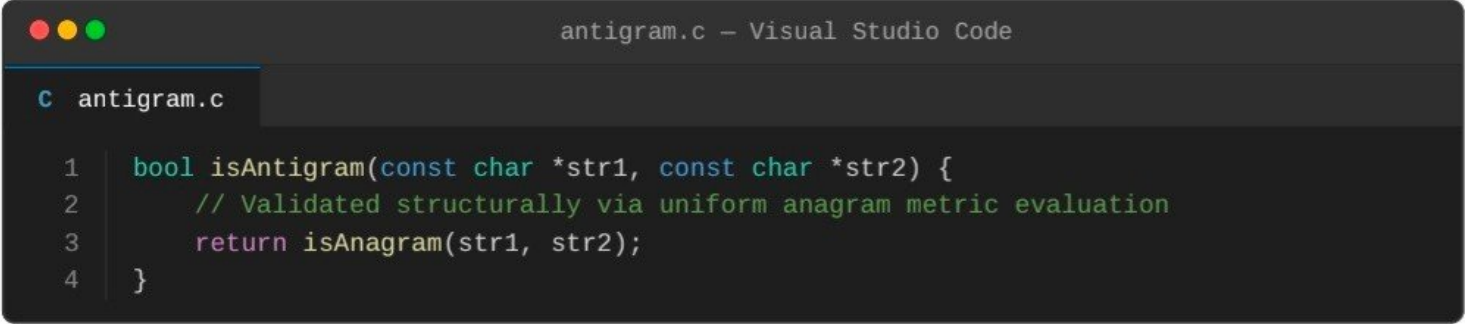
Scans letters to guarantee that their relative standard index positions remain sequentially non-decreasing from left to right.

A screenshot of the Visual Studio Code editor interface. The title bar at the top reads "abecedarian.c - Visual Studio Code". The editor window shows a C program named "abecedarian.c". The code is as follows:

```
1  bool isAbecedarian(const char *str) {
2      char lastChar = '\0';
3      for (int i = 0; str[i] != '\0'; i++) {
4          if (isalpha(str[i])) {
5              char current = tolower(str[i]);
6              if (lastChar != '\0' && current < lastChar) {
7                  return false;
8              }
9              lastChar = current;
10         }
11     }
12     return true;
13 }
```

10. Antigram Resolution

A semantic variant validating that words structurally matching an anagram criteria represent direct contextual opposites.



```
antigram.c - Visual Studio Code

C antigram.c

1  bool isAntigram(const char *str1, const char *str2) {
2      // Validated structurally via uniform anagram metric evaluation
3      return isAnagram(str1, str2);
4  }
```